

Jobsheet 2 – CSS3 Styling Dasar

Sub-CPMK: Mengimplementasikan styling dasar dengan CSS3.

Perubahan dari Jobsheet 1

- Tambah `assets/css/style.css` (box model, Flexbox untuk navbar, CSS Grid untuk kartu statistik Beranda).
- Semua halaman `.html` ditambahkan `<link rel="stylesheet">` ke `style.css` (path relatif menyesuaikan kedalaman folder).
- Struktur HTML **tidak diubah** – hanya tampilan.

Cara menjalankan

Buka `index.html` langsung di browser.

Catatan

- Kartu statistik di Beranda memakai `main section:nth-of-type(2)` sebagai grid 3 kolom.
- Class CSS bersifat generik (berbasis tag semantic + `nth-child`) agar bisa dipakai ulang di halaman Anggota tanpa duplikasi class.

Dokumentasi Jobsheet 2 – CSS3 Styling Dasar

Dokumentasi ini melanjutkan [dokumentasi jobsheet-01](#) dan ditujukan untuk mahasiswa yang baru belajar HTML/CSS. Jika kamu belum paham struktur HTML dasar (`header`, `nav`, `main`, `table`, `form`, dst.), sebaiknya baca dulu dokumentasi jobsheet-01 sebelum lanjut ke sini.

Apa yang Baru di Jobsheet 2?

Sesuai [README.md](#) jobsheet ini, **struktur HTML tidak** berubah sama sekali** dari jobsheet-01 – hanya ditambahkan satu file baru,** `assets/css/style.css`, dan satu baris `<link rel="stylesheet">` di tiap halaman HTML. Jadi seluruh dokumentasi jobsheet 2 ini fokus pada **CSS**: apa itu CSS, bagaimana cara kerjanya, dan penjelasan baris-per-baris file `style.css`.

Daftar Isi

1. [Konsep Dasar CSS](#)
2. [Apa yang Berubah di File HTML?](#)
3. [CSS: Reset & Gaya Dasar Body](#)
4. [CSS: Header & Navbar dengan Flexbox](#)
5. [CSS: Layout main & section](#)
6. [CSS: Kartu Statistik dengan CSS Grid](#)
7. [CSS: Styling Tabel](#)
8. [CSS: Styling Form](#)
9. [CSS: Footer](#)
10. [Rangkuman & Latihan Lanjutan](#)

Struktur Folder

```
jobsheet-02/
├── index.html
├── assets/
│   └── css/
│       └── style.css      # File baru di jobsheet ini
├── buku/
│   ├── list.html
│   └── tambah.html
├── anggota/
│   ├── list.html
│   └── tambah.html
├── README.md
└── dokumentasi/          # Folder dokumentasi ini
```

Silakan baca urut dari nomor 1 supaya konsepnya nyambung, atau langsung loncat ke bagian CSS yang ingin dipahami.

Dokumentasi Jobsheet 2 – CSS3 Styling Dasar

Dokumentasi ini melanjutkan [dokumentasi jobsheet-01](#) dan ditujukan untuk mahasiswa yang baru belajar HTML/CSS. Jika kamu belum paham struktur HTML dasar (`header`, `nav`, `main`, `table`, `form`, `dst.`), sebaiknya baca dulu dokumentasi jobsheet-01 sebelum lanjut ke sini.

Apa yang Baru di Jobsheet 2?

Sesuai [README.md](#) jobsheet ini, **struktur HTML tidak** berubah sama sekali** dari jobsheet-01 – hanya ditambahkan satu file baru,** `assets/css/style.css`, dan satu baris `<link rel="stylesheet">` di tiap halaman HTML. Jadi seluruh dokumentasi jobsheet 2 ini fokus pada **CSS**: apa itu CSS, bagaimana cara kerjanya, dan penjelasan baris-per-baris file `style.css`.

Daftar Isi

1. [Konsep Dasar CSS](#)
2. [Apa yang Berubah di File HTML?](#)
3. [CSS: Reset & Gaya Dasar Body](#)
4. [CSS: Header & Navbar dengan Flexbox](#)
5. [CSS: Layout `main` & `section`](#)
6. [CSS: Kartu Statistik dengan CSS Grid](#)
7. [CSS: Styling Tabel](#)
8. [CSS: Styling Form](#)
9. [CSS: Footer](#)
10. [Rangkuman & Latihan Lanjutan](#)

Struktur Folder

```
jobsheet-02/
├── index.html
├── assets/
│   ├── css/
│   │   └── style.css      # File baru di jobsheet ini
├── buku/
│   ├── list.html
│   └── tambah.html
└── anggota/
```

```
| └─ list.html
| └─ tambah.html
└─ README.md
    └─ dokumentasi/          # Folder dokumentasi ini
```

Silakan baca urut dari nomor 1 supaya konsepnya nyambung, atau langsung loncat ke bagian CSS yang ingin dipahami.

1. Konsep Dasar CSS

Sebelum membedah `style.css`, kenali dulu istilah-istilah CSS yang akan terus muncul di seluruh dokumentasi ini.

1.1 Apa itu CSS?

CSS (Cascading Style Sheets) adalah bahasa untuk mengatur **tampilan** elemen HTML — warna, ukuran, jarak, tata letak, dan sebagainya. Kalau HTML menentukan **struktur/isi** halaman (judul, paragraf, tabel, form), CSS menentukan **bagaimana rupanya** di layar.

1.2 Anatomi Satu Aturan CSS (CSS Rule)

```
header {
  background-color: #1d5b8a;
  color: #fff;
}
```

Bagian	Contoh	Fungsi
Selector	<code>header</code>	Menentukan elemen HTML mana yang diberi gaya ini. Di sini artinya "semua tag <code><header></code> ".
Deklarasi	<code>background-color: #1d5b8a;</code>	Satu aturan gaya, terdiri dari properti dan nilai , diakhiri titik koma <code>;</code> .
Properti	<code>background-color</code>	Aspek tampilan yang diatur (warna latar, ukuran font, dll).
Nilai	<code>#1d5b8a</code>	Nilai yang diberikan ke properti tersebut.

Bagian	Contoh	Fungsi
Blok deklarasi	{ ... }	Kurung kurawal yang membungkus satu atau lebih deklarasi untuk selector yang sama.

1.3 Menghubungkan CSS ke HTML

Ada 3 cara menghubungkan CSS ke HTML, tapi jobsheet ini memakai cara yang **paling direkomendasikan**: file CSS terpisah, dihubungkan lewat tag `<link>` di dalam `<head>`:

```
<link rel="stylesheet" href="assets/css/style.css">
```

- `rel="stylesheet"` — memberi tahu browser bahwa file yang di-link ini adalah lembar gaya (stylesheet).
- `href="..."` — lokasi file CSS-nya. Sama seperti tautan `<a href="...">` di [dokumentasi jobsheet-01](#), path ini **relatif** terhadap lokasi file HTML — lihat penjelasan lengkapnya di [bab 2](#).

Keuntungan memisahkan CSS ke file sendiri (dibanding menulis style langsung di setiap tag HTML): **satu file** `style.css` **dipakai bersama** oleh 5 halaman HTML sekaligus** — kalau warna tombol ingin diganti,** cukup ubah satu baris di `style.css`, otomatis berubah di semua halaman.

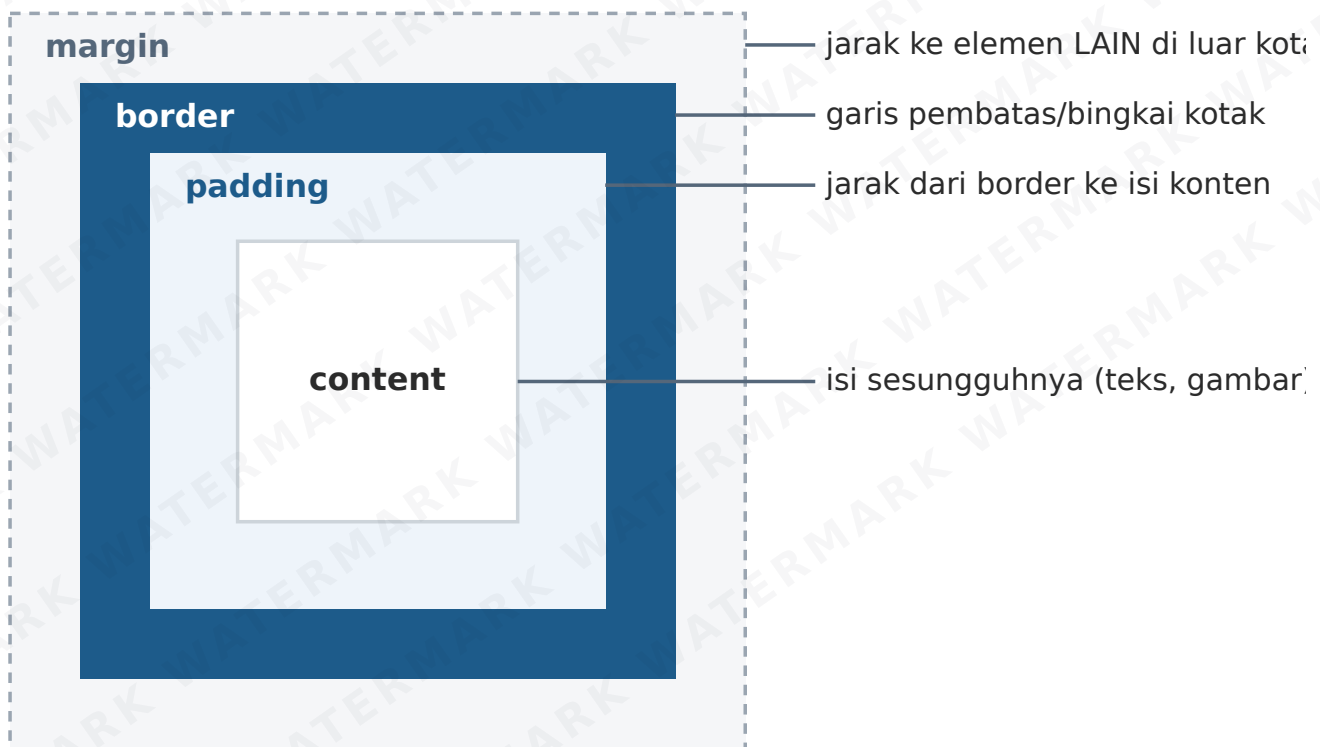
1.4 Jenis-Jenis Selector yang Dipakai di `style.css`

Jenis Selector	Contoh di <code>style.css</code>	Artinya
Tag/Elemen	<code>body</code> , <code>header</code> , <code>table</code>	Pilih semua elemen dengan tag tersebut.
Universal	<code>*</code>	Pilih semua elemen tanpa kecuali.
Turunan (descendant)	<code>header nav ul</code> , <code>td button</code>	Pilih elemen <code>ul</code> yang berada di dalam <code>nav</code> yang berada di dalam <code>header</code> . Urutan kata = urutan bersarang di HTML.
Pseudo-class	<code>a:hover</code> , <code>tbody tr:hover</code>	Pilih elemen pada kondisi/state tertentu — <code>:hover</code> berarti "saat kursor mouse berada di atas elemen ini".

Jenis Selector	Contoh di <code>style.css</code>	Artinya
Pseudo-class posisi	<code>tbody tr:nth-child(even),</code> <code>main section:nth-of-type(2)</code>	Pilih elemen berdasarkan urutan/posisinya di antara saudara-saudaranya. Dibahas detail di bab 6 dan bab 7 .
Atribut	<code>form button[type="submit"]</code>	Pilih elemen <code>button</code> yang atributnya <code>type</code> bernilai <code>"submit"</code> .

1.5 Kotak Setiap Elemen HTML: Box Model

Setiap elemen HTML di browser sebenarnya adalah sebuah **kotak (box)**, tersusun dari 4 lapisan dari dalam ke luar:



- `margin` — jarak kosong **di luar** kotak, ke elemen tetangga.
- `border` — garis di tepi kotak.
- `padding` — jarak kosong **di dalam** kotak, antara border dan isi.
- `content` — konten sebenarnya (teks, gambar, dll).

Properti `box-sizing: border-box;` (dipakai di baris pertama `style.css`, lihat [bab 3](#)) mengubah cara lebar/tinggi elemen dihitung. Ini konsep penting yang dijelaskan detail di bab berikutnya.

1.6 Satuan Ukuran yang Dipakai

Di `style.css` kamu akan menemukan beberapa satuan ukuran berbeda:

Satuan	Contoh	Artinya
px (pixel)	<code>border-radius: 8px;</code>	Ukuran tetap/absolut , tidak berubah mengikuti apa pun.
rem	<code>padding: 1rem 1.5rem;</code>	Ukuran relatif terhadap ukuran font akar (<code><html></code>), biasanya <code>1rem = 16px</code> secara default. Lebih fleksibel karena ikut menyesuaikan jika pengguna mengubah ukuran font browser.
%	<code>width: 100%;</code>	Ukuran relatif terhadap elemen induknya (misalnya lebar 100% dari lebar kotak pembungkusnya).
fr (fraction)	<code>grid-template-columns: repeat(3, 1fr);</code>	Satuan khusus CSS Grid: "1 bagian pecahan" dari ruang yang tersedia. Dibahas di bab 6 .

1.7 Warna dalam Format Hex

Warna di `style.css` banyak ditulis dalam format **hex** (heksadesimal), contoh `#1d5b8a` atau `#fff`:

- Format `#RRGGBB` — dua digit pertama untuk merah (Red), dua berikutnya hijau (Green), dua terakhir biru (Blue). Contoh `#1d5b8a` adalah biru tua (komponen biru paling dominan).
- `#fff` adalah bentuk singkat dari `#ffffff` (putih penuh) — kalau ketiga pasangan digitnya sama, boleh ditulis 3 digit saja.

Dengan bekal istilah-istilah di atas, sekarang kamu siap membaca penjelasan file `style.css` secara detail mulai bab 3.

Lanjut ke: [Apa yang Berubah di File HTML?](#)

2. Apa yang Berubah di File HTML?

Kabar baik: kalau kamu sudah paham [dokumentasi HTML jobsheet-01](#), kamu **tidak perlu belajar ulang** struktur HTML-nya di jobsheet ini. Semua `header`, `nav`, `main`, `section`, `article`,

table, form, footer di 5 file HTML **persis sama** dengan jobsheet-01.

Satu-satunya perubahan di setiap file HTML adalah **satu baris baru** di dalam `<head>` :

```
<link rel="stylesheet" href="assets/css/style.css">
```

(penjelasan tag `<link>` ada di [konsep dasar CSS §1.3](#))

2.1 Path CSS Berbeda-beda per File

Karena file `style.css` disimpan di satu lokasi (`jobsheet-02/assets/css/style.css`), sedangkan file-file HTML tersebar di beberapa folder dengan kedalaman berbeda, nilai `href` pada tag `<link>` harus disesuaikan:

File HTML	Lokasi File	href yang Dipakai
<code>index.html</code>	folder root (<code>jobsheet-02/</code>)	<code>assets/css/style.css</code>
<code>buku/list.html</code>	dalam folder <code>buku/</code>	<code>../assets/css/style.css</code>
<code>buku/tambah.html</code>	dalam folder <code>buku/</code>	<code>../assets/css/style.css</code>
<code>anggota/list.html</code>	dalam folder <code>anggota/</code>	<code>../assets/css/style.css</code>
<code>anggota/tambah.html</code>	dalam folder <code>anggota/</code>	<code>../assets/css/style.css</code>

Polanya sama seperti path pada `<a href="...">` di menu navigasi ([lihat penjelasan jobsheet-01 §1.5](#)): `../` berarti "naik satu folder ke atas" sebelum masuk ke `assets/css/style.css`.

Kesalahan paling umum pemula saat menambahkan CSS ke banyak halaman di folder berbeda adalah **lupa menyesuaikan jumlah `../`** — akibatnya file CSS "tidak ke-load" dan halaman tetap tampil polos tanpa ada pesan error yang jelas di layar (biasanya baru terlihat errornya di tab

- Console*/*Network* pada DevTools browser).

2.2 Kenapa Struktur HTML Sengaja Tidak Diubah?

Ini keputusan desain yang penting dipahami: CSS **seharusnya** bisa mengubah total tampilan sebuah halaman **tanpa** perlu mengubah struktur HTML-nya. Prinsip ini disebut **separation of concerns** (pemisahan tanggung jawab) — HTML mengurus struktur/makna konten, CSS mengurus tampilan. Jobsheet ini sengaja dirancang begitu supaya kamu bisa

membandingkan langsung: font "Segoe UI", warna biru pada header, navbar sejajar horizontal, kartu statistik yang tersusun rapi 3 kolom — semua itu **murni hasil CSS**, bukan HTML baru.

Coba bandingkan tangkapan layar `index.html` jobsheet-01 (tanpa CSS, tampilan default browser, semua elemen bertumpuk vertikal polos) dengan `index.html` jobsheet-02 (dengan CSS) untuk melihat sendiri betapa besarnya efek CSS terhadap tampilan, padahal HTML-nya identik.

Lanjut ke: [CSS: Reset & Gaya Dasar Body](#)

3. CSS: Reset & Gaya Dasar Body

Ini bagian **paling atas** file `style.css` — fondasi yang memengaruhi seluruh halaman.

3.1 CSS Reset dengan Selector Universal *

```
/* ===== Reset & Base ===== */
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
```

- `*` adalah **universal selector** — artinya aturan ini berlaku untuk **semua elemen HTML** tanpa kecuali (`div`, `p`, `h1`, `button`, `dst`).
- `margin: 0; padding: 0;` menghapus jarak bawaan (default) yang secara otomatis diberikan browser ke banyak elemen (misalnya `<h1>`, `<p>`, `<ul>` punya margin bawaan yang berbeda-beda tiap browser). Ini disebut **CSS reset** — "menyamakan garis start" supaya tampilan konsisten di semua browser, sebelum diatur ulang sesuai keinginan kita sendiri di aturan-aturan berikutnya.
- `box-sizing: border-box;` — ini yang paling penting untuk dipahami: secara default (`content-box`), kalau kamu mengatur `width: 200px` lalu menambahkan `padding: 20px`, lebar **total** kotak yang tampil di layar jadi 240px ($200 + 20 + 20$ kiri-kanan) — membingungkan untuk pemula. Dengan `border-box`, `width: 200px` berarti lebar **total** kotak (termasuk padding & border) tetap 200px — padding "dimakan" dari dalam. Inilah alasan hampir semua developer CSS modern selalu menambahkan aturan `box-sizing: border-box;` di awal stylesheet mereka.

Komentar `/* ===== Reset & Base ===== */` di atasnya adalah **komentar CSS**** (diapit `/* ... */`), tidak memengaruhi tampilan sama sekali —** hanya catatan untuk programmer supaya file mudah dinavigasi (bandingkan dengan komentar HTML `<!-- ... -->`).

3.2 Gaya Dasar <body>

```
body {  
  font-family: "Segoe UI", Arial, sans-serif;  
  color: #2b2b2b;  
  background-color: #f5f6f8;  
  line-height: 1.5;  
}
```

Properti	Nilai	Penjelasan
font-family	"Segoe UI", Arial, sans-serif	Daftar font berurutan berdasarkan prioritas : browser mencoba pakai "Segoe UI" dulu; kalau font itu tidak tersedia di komputer pengguna, coba "Arial"; kalau itu juga tidak ada, pakai font <code>sans-serif</code> generik apa pun yang tersedia di sistem. Selalu sertakan font generik di akhir daftar sebagai jaminan cadangan (<i>fallback</i>).
color	#2b2b2b	Warna teks , abu-abu sangat gelap (hampir hitam, tapi tidak hitam pekat <code>#000</code> — supaya lebih lembut di mata).
background-color	#f5f6f8	Warna latar belakang seluruh halaman, abu-abu sangat muda.
line-height	1.5	Jarak antar baris teks, 1.5 kali tinggi font. Tanpa satuan, artinya kelipatan dari ukuran font elemen itu sendiri. Nilai ini membuat teks lebih nyaman dibaca dibanding default browser (biasanya sekitar 1.2).

Karena aturan ini ditempel di selector `body`, dan hampir semua elemen lain berada **di dalam** `<body>`, properti `font-family`, `color`, dan `line-height` ini otomatis "diturunkan" (*inherited*) ke elemen anak seperti `<p>`, `<h2>`, `<td>`, dst., kecuali elemen tersebut diberi aturan CSS lain yang menyimpannya secara spesifik (misalnya `header h1` punya `font-size` sendiri, dijelaskan di [bab 4](#)).

3.3 Gaya Tautan (<a>)

```
a {  
  color: #1d5b8a;  
  text-decoration: none;  
}
```

```
a:hover {  
  text-decoration: underline;  
}
```

- `a { color: #1d5b8a; }` — semua tautan diberi warna biru (senada dengan warna header, lihat [bab 4](#)), menggantikan warna biru bawaan browser.
- `text-decoration: none;` — menghilangkan garis bawah otomatis yang biasanya menyertai tautan, supaya tampilan lebih bersih.
- `a:hover { text-decoration: underline; }` — ini **pseudo-class** (lihat [konsep dasar §1.4](#)): garis bawah baru muncul **saat kursor mouse diarahkan (hover)** ke atas tautan. Ini memberi umpan balik visual ke pengguna bahwa teks tersebut bisa diklik, tanpa membuat tampilan berantakan saat tidak sedang disentuh kursor.

Lanjut ke: [CSS: Header & Navbar dengan Flexbox](#)

4. CSS: Header & Navbar dengan Flexbox

Bagian ini adalah pengenalan pertama ke **Flexbox** — salah satu sistem tata letak (*layout*) terpenting di CSS modern.

4.1 Kode CSS

```
/* ===== Header & Navbar (Flexbox) ===== */  
header {  
  background-color: #1d5b8a;  
  color: #fff;  
  padding: 1rem 1.5rem;  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  flex-wrap: wrap;  
}  
  
header h1 {  
  font-size: 1.4rem;  
}  
  
header nav ul {  
  list-style: none;  
  display: flex;  
  gap: 1.25rem;  
}  
  
header nav a {  
  color: #fff;
```

```
font-weight: 500;
}
```

4.2 Apa itu Flexbox?

Secara default, elemen HTML seperti `<h1>` dan `<nav>` di dalam `<header>` akan tersusun **vertikal** (bertumpuk ke bawah), karena keduanya adalah elemen *block*. Padahal yang kita inginkan: judul "SIMPUS-Mini" di kiri, menu navigasi di kanan, **sejajar dalam satu baris horizontal**. Di sinilah Flexbox berperan — sistem tata letak 1 dimensi yang mengatur bagaimana elemen-elemen anak tersusun di dalam sebuah kotak pembungkus.

4.3 Mengaktifkan Flexbox: `display: flex`

```
header {
  display: flex;
  ...
}
```

Menuliskan `display: flex;` pada `<header>` mengubah `<header>` menjadi **flex container**, dan **semua anak langsung**-nya (di sini: `<h1>` dan `<nav>`) otomatis menjadi **flex item** yang tersusun sejajar secara horizontal (default-nya), bukan bertumpuk vertikal lagi.

4.4 Mengatur Posisi Flex Item

Properti	Nilai	Efek
<code>align-items:</code> <code>center;</code>	—	Menyejajarkan flex item secara vertikal di tengah sumbu silang (<i>cross axis</i>) — supaya <code><h1></code> dan <code><nav></code> sama-sama center secara vertikal meski tingginya berbeda.
<code>justify-content:</code> <code>space-between;</code>	—	Mengatur jarak antar flex item pada sumbu utama (<i>main axis</i> , horizontal). <code>space-between</code> mendorong item pertama menempel di ujung kiri , item terakhir menempel di ujung kanan , dan sisa ruang kosong dibagi rata di antaranya. Inilah yang membuat <code><h1></code> di kiri dan <code><nav></code> di kanan header.
<code>flex-wrap:</code> <code>wrap;</code>	—	Mengizinkan flex item pindah ke baris baru kalau ruang tidak cukup (misalnya di layar HP yang sempit), daripada dipaksa muat dalam satu baris sampai terpotong/menyempit berlebihan.

4.5 Padding pada Header

```
padding: 1rem 1.5rem;
```

CSS memperbolehkan menulis 2 nilai sekaligus pada properti `padding` (atau `margin`): nilai **pertama untuk atas-bawah**, nilai **kedua untuk kiri-kanan****. Jadi `1rem 1.5rem` berarti padding atas & bawah `1rem`,** padding kiri & kanan `1.5rem`. Ini cara singkat menulis 4 nilai berbeda tanpa harus menulis `padding-top`, `padding-right`, `padding-bottom`, `padding-left` satu per satu.

4.6 Flexbox Bertingkat: Navbar di Dalam Header

Menariknya, Flexbox dipakai **dua kali bertingkat** di sini — sekali di `<header>` (menyusun `h1` & `nav` horizontal), dan sekali lagi di dalam `<nav>` `<ul>` untuk menyusun **item-item menu**:

```
header nav ul {  
  list-style: none;  
  display: flex;  
  gap: 1.25rem;  
}
```

- `header nav ul` — descendant selector (lihat [konsep dasar §1.4](#)): memilih elemen `<ul>` yang berada di dalam `<nav>` yang berada di dalam `<header>`.
- `list-style: none;` — menghapus bulatan/titik (bullet point) bawaan daftar `<ul>`, karena di navbar kita tidak butuh tampilan bullet list.
- `display: flex;` — mengubah `<ul>` (yang isinya `<li>` per item menu) jadi flex container juga, sehingga semua `<li>` tersusun sejajar horizontal alih-alih bertumpuk vertikal seperti daftar biasa.
- `gap: 1.25rem;` — properti modern Flexbox/Grid untuk memberi **jarak** seragam** antar flex item, tanpa perlu mengatur `margin` manual di** tiap item satu-satu.

```
header nav a {  
  color: #fff;  
  font-weight: 500;  
}
```

Ini menimpa (*override*) warna tautan bawaan (`a { color: #1d5b8a; }` dari [bab 3](#)) **khusus untuk** tautan di dalam navbar** — supaya teks menu berwarna putih (`#fff`),** kontras dengan latar belakang header yang biru tua. `font-weight: 500` membuat teksnya sedikit lebih tebal dari normal (`400`) tapi belum setebal `bold` (`700`).

4.7 Kenapa Selector `header nav a` Lebih "Menang" daripada `a`?

Ini contoh sederhana konsep **spesifisitas CSS** (*specificity*): selector yang lebih spesifik/detail (header nav a, menyebutkan 3 tag bersarang) mengalahkan selector yang lebih umum (a, hanya 1 tag), **tanpa memedulikan urutan penulisan di file**. Jadi meskipun aturan a { color: #1d5b8a; } ditulis lebih dulu di file, aturan header nav a tetap yang menentukan warna tautan di dalam navbar, karena lebih spesifik.

Lanjut ke: [CSS: Layout main & section](#)

5. CSS: Layout main & section

Bagian ini mengatur lebar konten utama halaman dan tampilan setiap <section> menjadi "kartu" (card) putih yang terangkat dari latar belakang abu-abu.

5.1 Kode CSS

```
/* ===== Main Layout ===== */
main {
  max-width: 1000px;
  margin: 2rem auto;
  padding: 0 1.5rem;
}

section {
  background-color: #fff;
  border-radius: 8px;
  padding: 1.5rem;
  margin-bottom: 1.5rem;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);
}

section h2 {
  margin-bottom: 1rem;
  color: #1d5b8a;
}
```

5.2 Membatasi Lebar & Menengahkan Konten (main)

```
main {
  max-width: 1000px;
  margin: 2rem auto;
  padding: 0 1.5rem;
}
```

- `max-width: 1000px;` — lebar `<main>` **tidak akan pernah melebihi** 1000px, meskipun layar monitor jauh lebih lebar. Ini praktik umum supaya teks/konten tidak melebar tak wajar dan sulit dibaca di layar besar (bayangkan membaca paragraf yang lebarnya sama dengan layar monitor 27 inci — matanya cepat lelah).
- `margin: 2rem auto;` — ingat dari [bab 4](#), 2 nilai berarti atas-bawah lalu kiri-kanan. Di sini: margin atas-bawah `2rem`, dan margin kiri-kanan `auto`. Nilai `auto` pada margin kiri-kanan adalah **trik klasik CSS** untuk **menengahkan elemen secara horizontal** — browser otomatis membagi rata sisa ruang kosong di kiri dan kanan `<main>` (yang lebarnya dibatasi `max-width: 1000px` tadi), sehingga kontennya selalu berada di tengah layar.
- `padding: 0 1.5rem;` — padding atas-bawah `0`, padding kiri-kanan `1.5rem`. Ini memberi sedikit jarak dari tepi layar supaya konten tidak terlalu mepet ke pinggir, terutama di layar sempit (HP/tablet) di mana `max-width: 1000px` belum berlaku (layarnya memang lebih sempit dari itu).

5.3 Kartu Putih untuk Setiap `<section>`

```
section {
  background-color: #fff;
  border-radius: 8px;
  padding: 1.5rem;
  margin-bottom: 1.5rem;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);
}
```

Properti	Efek
<code>background-color: #fff;</code>	Latar putih, kontras dengan latar halaman abu-abu (<code>#f5f6f8</code> dari <code>body</code> , bab 3) — inilah yang membuat tiap <code><section></code> terlihat sebagai "kartu" terpisah.
<code>border-radius: 8px;</code>	Membuat sudut kotak melengkung/membulat sebesar 8px, bukan siku tajam. Efek visual yang umum dipakai untuk kesan modern dan lembut.
<code>padding: 1.5rem;</code>	Satu nilai saja berarti jarak yang sama di keempat sisi (atas, bawah, kiri, kanan) — memberi ruang napas antara tepi kartu dan isinya.
<code>margin-bottom: 1.5rem;</code>	Jarak di bawah tiap kartu, memisahkannya dari kartu/elemen berikutnya di bawahnya.
<code>box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);</code>	Memberi bayangan halus di sekeliling kartu, memberi kesan kartu "terangkat" sedikit dari latar belakang. Nilainya: <code>0</code> (bayangan tidak bergeser ke kanan/kiri), <code>1px</code> (bergeser 1px ke bawah), <code>3px</code> (blur/kabur

Properti	Efek
	sejauh 3px), dan <code>rgba(0, 0, 0, 0.08)</code> (warna hitam dengan opacity/transparansi hanya 8% — bayangan sangat samar/halus, tidak mencolok).

Catatan tentang `rgba()` : ini format warna alternatif dari hex (`#1d5b8a`) yang dijelaskan di [konsep dasar §1.7](#). `rgba(R, G, B, A)` menulis warna dalam angka Merah-Hijau-Biru (0–255) ditambah **Alpha** (tingkat transparansi, dari 0 = tembus pandang total sampai 1 = pekat penuh). Format ini dipilih di sini **karena butuh** transparansi** untuk bayangan, sesuatu yang tidak bisa dilakukan format** hex biasa.

5.4 Judul di Dalam Section

```
section h2 {
  margin-bottom: 1rem;
  color: #1d5b8a;
}
```

Setiap `<h2>` yang berada di dalam `<section>` (semua judul section di seluruh halaman, seperti "Ringkasan", "Daftar Buku", "Tambah Anggota") diberi jarak `1rem` di bawahnya (memisahkan judul dari konten di bawahnya) dan warna biru senada dengan header (`#1d5b8a`) — memberi kesan "warna tema" yang konsisten di seluruh aplikasi.

Lanjut ke: [CSS: Kartu Statistik dengan CSS Grid](#)

6. CSS: Kartu Statistik dengan CSS Grid

Bagian ini memperkenalkan **CSS Grid**, sistem layout lain (selain Flexbox) yang dipakai khusus untuk menyusun 3 kartu statistik di Beranda (Total Buku, Total Anggota, Sedang Dipinjam) menjadi 3 kolom sejajar.

6.1 Mengingat Kembali HTML-nya

Dari [dokumentasi HTML jobsheet-01](#), `index.html` punya struktur `<main>` berisi **2 buah** `<section>`:

```
<main>
  <section>                                <!-- section ke-1: sambutan -->
    <h2>Selamat Datang...</h2>
    <p>...</p>
  </section>
```

```

<section>                                <!-- section ke-2: ringkasan statistik -->
  <h2>Ringkasan</h2>
  <article>...Total Buku...</article>
  <article>...Total Anggota...</article>
  <article>...Sedang Dipinjam...</article>
</section>
</main>

```

Section **kedua** inilah yang berisi 3 `<article>` kartu statistik, dan inilah yang jadi target CSS Grid.

6.2 Kode CSS

```

/* ===== Kartu Statistik (CSS Grid) ===== */
main section:nth-of-type(2) {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}

main section:nth-of-type(2) article {
  background-color: #eef4fa;
  border-radius: 8px;
  padding: 1.25rem;
  text-align: center;
}

main section:nth-of-type(2) article h3 {
  font-size: 0.95rem;
  color: #55677a;
  margin-bottom: 0.5rem;
}

main section:nth-of-type(2) article p {
  font-size: 1.8rem;
  font-weight: 700;
  color: #1d5b8a;
}

```

6.3 Selector `:nth-of-type(2)` — Memilih "Section Kedua Saja"

```
main section:nth-of-type(2) { ... }
```

- Perhatikan HTML `index.html` punya 2 elemen `<section>` yang sama jenis tag-nya, tanpa `class` atau `id` pembeda apa pun di antaranya.

- `:nth-of-type(2)` adalah pseudo-class yang memilih elemen berdasarkan **urutan kemunculannya** di antara saudara-saudara dengan **tag yang sama****. Jadi `section:nth-of-type(2)` artinya "section yang urutannya** ke-2 di antara semua elemen `<section>` yang sejenis".
- Digabung dengan `main` di depannya (descendant selector), maka `main section:nth-of-type(2)` berarti "section kedua yang berada di dalam `<main>`" — yaitu section "Ringkasan" (bukan section "Selamat Datang" yang urutannya pertama).

Ini lah yang dimaksud catatan di [README.md](#) jobsheet ini:

- "Kartu statistik di Beranda memakai `main section:nth-of-type(2)` sebagai grid 3 kolom"* — dan alasan kenapa pendekatan ini dipilih (ketimbang menambahkan `class` khusus di HTML) dijelaskan di [§6.7](#).

6.4 Mengaktifkan Grid: `display: grid`

```
display: grid;
grid-template-columns: repeat(3, 1fr);
gap: 1rem;
```

- `display: grid;` — mengubah section ini menjadi **grid container**. Semua anak langsungnya (di sini: 3 elemen `<article>`) otomatis menjadi **grid item** yang disusun mengikuti kolom/baris yang didefinisikan.
- `grid-template-columns: repeat(3, 1fr);` — mendefinisikan **3 kolom**, masing-masing lebar `1fr`. `repeat(3, 1fr)` adalah singkatan dari menulis `1fr 1fr 1fr` tiga kali. Ingat satuan `fr` dari [konsep dasar §1.6](#): karena ketiga kolom sama-sama `1fr`, ruang yang tersedia dibagi **rata** sama besar** ke 3 kolom itu.**
- `gap: 1rem;` — jarak seragam antar kolom (dan antar baris jika ada lebih dari satu baris), sama seperti fungsi `gap` di Flexbox navbar ([bab 4](#)).

6.5 Flexbox vs Grid — Bedanya Apa?

Pertanyaan wajar dari pemula: kenapa navbar pakai Flexbox tapi kartu statistik pakai Grid, padahal sama-sama "menyusun elemen sejajar"?

	Flexbox	CSS Grid
Dimensi	1 dimensi (baris atau kolom)	2 dimensi (baris dan kolom sekaligus)

	Flexbox	CSS Grid
Cocok untuk	Menyusun jumlah item yang tidak tetap/fleksibel dalam satu baris/kolom (seperti menu navbar yang jumlah itemnya bisa berubah)	Menyusun tata letak kisi/kotak-kotak dengan ukuran kolom yang presisi ditentukan sejak awal (seperti kartu statistik yang selalu 3 kolom sejajar)

Di jobsheet ini, navbar dipilih Flexbox karena hanya perlu menyusun item menu sejajar horizontal (1 arah), sedangkan kartu statistik dipilih Grid karena butuh kontrol eksplisit "3 kolom sama lebar" — kasus paling umum di mana Grid lebih ringkas dipakai dibanding Flexbox.

6.6 Styling Isi Tiap Kartu (<article>)

```
main section:nth-of-type(2) article {
  background-color: #eef4fa;
  border-radius: 8px;
  padding: 1.25rem;
  text-align: center;
}
```

Setiap <article> (kartu individual) diberi latar biru sangat muda (#eef4fa, beda dari latar putih section itu sendiri — lihat [bab 5](#) — supaya kartu terlihat sebagai "kotak di dalam kotak"), sudut membulat, padding, dan teks rata tengah (text-align: center).

```
main section:nth-of-type(2) article h3 {
  font-size: 0.95rem;
  color: #55677a;
  margin-bottom: 0.5rem;
}

main section:nth-of-type(2) article p {
  font-size: 1.8rem;
  font-weight: 700;
  color: #1d5b8a;
}
```

- <h3> (judul kecil, misal "Total Buku") dibuat **lebih kecil** (0.95rem, lebih kecil dari ukuran teks normal 1rem) dan berwarna abu-kebiruan (#55677a) — karena ini hanya label, bukan fokus utama.
- <p> (angkanya, misal "12") dibuat jauh **lebih besar** (1.8rem), tebal (font-weight: 700 = bold), dan berwarna biru tema (#1d5b8a) — karena inilah informasi utama yang ingin ditonjolkan ke pengguna. Perbedaan ukuran dan bobot font antara label kecil dan angka

besar ini adalah teknik umum dalam desain UI yang disebut **hierarki visual**** (*visual hierarchy*) – mengarahkan mata pengguna ke informasi** terpenting lebih dulu.

6.7 Kenapa Tidak Pakai `class` Saja?

Sesuai catatan di [README.md](#) jobsheet ini: *"Class CSS bersifat generik (berbasis tag semantic + nth-child) agar bisa dipakai ulang di halaman Anggota tanpa duplikasi class."* Artinya, alih-alih menambahkan `class="stat-grid"` secara manual di HTML lalu menulis `.stat-grid { display: grid; ... }` di CSS, jobsheet ini sengaja memilih pendekatan selector berbasis **struktur & posisi** (`main section:nth-of-type(2)`). Keuntungannya: HTML tetap bersih tanpa tambahan atribut `class` apa pun, dan gaya yang sama otomatis berlaku di halaman lain asalkan strukturnya serupa. Kekurangannya (yang perlu kamu sadari sebagai pembelajar): kalau urutan `<section>` di HTML berubah (misalnya section "Ringkasan" dipindah jadi section pertama), gaya Grid ini **ikut salah sasaran** karena `:nth-of-type(2)` menempel pada posisi, bukan makna. Ini adalah trade-off (kompromi) yang wajar dipahami saat kamu mulai membuat keputusan desain CSS sendiri di kemudian hari.

Lanjut ke: [CSS: Styling Tabel](#)

7. CSS: Styling Tabel

Bagian ini mempercantik tabel `<table>` di `buku/list.html` dan `anggota/list.html` (lihat strukturnya di [dokumentasi HTML jobsheet-01 §3.2](#)).

7.1 Kode CSS

```
/* ===== Tabel ===== */
table {
  width: 100%;
  border-collapse: collapse;
}

th, td {
  text-align: left;
  padding: 0.65rem 0.75rem;
  border-bottom: 1px solid #e2e6ea;
}

thead {
  background-color: #1d5b8a;
  color: #fff;
}
```

```
tbody tr:nth-child(even) {
    background-color: #f7f9fb;
}

tbody tr:hover {
    background-color: #eef4fa;
}

td button {
    padding: 0.35rem 0.7rem;
    margin-right: 0.35rem;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-size: 0.85rem;
}

td button:first-of-type {
    background-color: #f0ad4e;
    color: #fff;
}

td button:last-of-type {
    background-color: #d9534f;
    color: #fff;
}
```

7.2 Lebar Tabel & border-collapse

```
table {
    width: 100%;
    border-collapse: collapse;
}
```

- `width: 100%;` — tabel melebar memenuhi seluruh lebar kotak pembungkusnya (kartu `<section>`, lihat [bab 5](#)), bukan hanya selebar isi terpanjangnya seperti perilaku default tabel.
- `border-collapse: collapse;` — mengatur agar garis batas (`border`) antar sel tabel **digabung menjadi satu garis tipis**, bukan tampil sebagai dua garis terpisah dengan jarak di antaranya (perilaku default `border-collapse: separate`). Ini yang membuat garis pembatas baris di tabel terlihat rapi & tipis, bukan bergaris ganda.

7.3 Sel Header (`th`) dan Sel Data (`td`)

```
th, td {
  text-align: left;
  padding: 0.65rem 0.75rem;
  border-bottom: 1px solid #e2e6ea;
}
```

- Selector `th, td` (dipisah koma) berarti aturan yang sama diterapkan ke **kedua** jenis sel sekaligus — cara menyingkat penulisan kalau dua selector berbeda butuh gaya identik, daripada menulis dua blok terpisah.
- `text-align: left;` — teks rata kiri. Ini sengaja ditulis eksplisit karena `<th>` secara default punya perataan tengah (`center`) di kebanyakan browser — aturan ini menyamakan perataan `th` dengan `td` supaya kolom judul dan data sejajar rapi.
- `border-bottom: 1px solid #e2e6ea;` — garis tipis abu-abu muda **hanya** di bagian bawah** tiap sel, memisahkan baris satu dengan baris di** bawahnya (mirip garis pada tabel excel yang sederhana, tanpa garis vertikal antar kolom).

7.4 Header Tabel Berwarna

```
thead {
  background-color: #1d5b8a;
  color: #fff;
}
```

Baris judul kolom (`<thead>`, lihat [dokumentasi HTML jobsheet-01](#)) diberi latar biru tema yang sama dengan header halaman, dengan teks putih — menegaskan secara visual bahwa baris ini adalah "judul kolom", berbeda dari baris-baris data di bawahnya.

7.5 Baris Selang-seling & Efek Hover

```
tbody tr:nth-child(even) {
  background-color: #f7f9fb;
}

tbody tr:hover {
  background-color: #eef4fa;
}
```

- `tbody tr:nth-child(even)` — mirip `:nth-of-type(2)` di [bab 6](#), tapi `:nth-child(even)` memilih **semua baris bernomor genap** (baris ke-2, ke-4, ke-6, dst.) di dalam `<tbody>`, lalu diberi warna latar abu sangat muda (`#f7f9fb`) yang sedikit berbeda dari latar putih section. Efek ini disebut **zebra stripes** (garis-garis mirip zebra) — teknik desain tabel klasik yang

membantu mata pembaca **mengikuti satu baris data secara horizontal** tanpa "tersasar" ke baris lain, terutama pada tabel dengan banyak kolom.

- `tbody tr:hover` — pseudo-class `:hover`, sama konsepnya dengan `a:hover` di [bab 3](#): saat kursor mouse diarahkan ke atas sebuah baris, latar baris itu berubah jadi biru sangat muda (`#eef4fa`, warna yang sama dengan latar kartu statistik di [bab 6](#)), menandakan baris mana yang sedang "disorot" — berguna terutama saat tabel punya banyak baris.

Catatan penting: aturan `:hover` ini diletakkan **setelah** aturan `:nth-child(even)` di file CSS.

Karena keduanya punya spesifisitas yang sama (sama-sama satu pseudo-class ditambah `tr`), urutan penulisan di file **menentukan** aturan mana yang menang ketika kursor berada di atas baris genap — dan karena `:hover` ditulis belakangan, warnanya (`#eef4fa`) yang tampil, menimpa warna zebra stripe (`#f7f9fb`) sementara kursor ada di situ.

7.6 Tombol Aksi (Edit & Hapus)

```
td button {
  padding: 0.35rem 0.7rem;
  margin-right: 0.35rem;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  font-size: 0.85rem;
}
```

Ini gaya dasar yang berlaku untuk **kedua** tombol (Edit dan Hapus) di kolom "Aksi" (lihat HTML-nya di [dokumentasi jobsheet-01](#)):

- `border: none;` — menghapus garis bingkai/border bawaan tombol HTML (biasanya abu-abu 3D ala tombol Windows lama).
- `border-radius: 4px;` — sudut sedikit membulat.
- `cursor: pointer;` — mengubah bentuk kursor mouse menjadi ikon "tangan menunjuk" saat berada di atas tombol, memberi sinyal visual bahwa elemen ini bisa diklik (tombol HTML biasanya sudah begini secara default, tapi baris ini menjamin konsistensi meski gaya lain berubah).
- `margin-right: 0.35rem;` — jarak di sebelah kanan setiap tombol, agar tombol "Edit" dan "Hapus" tidak saling menempel.

```
td button:first-of-type {
  background-color: #f0ad4e;
  color: #fff;
}

td button:last-of-type {
  background-color: #d9534f;
```

```
color: #fff;
}
```

Dua aturan ini membedakan warna **berdasarkan urutan tombol**, bukan berdasarkan teks/isi tombolnya:

- `:first-of-type` — tombol **pertama** di antara tombol-tombol sejenis di dalam sel yang sama → diberi warna **oranye/kuning** (`#f0ad4e`). Karena di HTML tombol pertama selalu "Edit", maka secara visual tombol Edit selalu oranye.
- `:last-of-type` — tombol **terakhir** → diberi warna **merah** (`#d9534f`), sesuai konvensi umum antarmuka (UI) bahwa warna merah menandakan aksi berisiko/merusak seperti "Hapus".

Ini contoh lain dari pola yang sama dengan [kartu statistik di bab 6](#): CSS "menebak" peran elemen dari **posisinya** dalam HTML (tombol pertama = Edit, tombol terakhir = Hapus), bukan dari `class` khusus seperti `class="btn-edit"` atau `class="btn-hapus"`. Cara ini ringkas, tapi mengasumsikan urutan tombol di HTML **tidak akan berubah** — kalau suatu saat kamu menambahkan tombol ketiga (misalnya "Detail") di antara Edit dan Hapus, warnanya bisa jadi tidak sesuai harapan lagi.

Lanjut ke: [CSS: Styling Form](#)

8. CSS: Styling Form

Bagian ini mempercantik form di `buku/tambah.html` dan `anggota/tambah.html` (lihat strukturnya di [dokumentasi HTML jobsheet-01](#)).

8.1 Kode CSS

```
/* ===== Form ===== */
form p {
  margin-bottom: 1rem;
}

form label {
  display: block;
  margin-bottom: 0.35rem;
  font-weight: 600;
  color: #444;
}

form input,
form select {
  width: 100%;
```

```

max-width: 400px;
padding: 0.55rem 0.7rem;
border: 1px solid #cdd4da;
border-radius: 4px;
font-size: 1rem;
}

form button[type="submit"] {
  background-color: #1d5b8a;
  color: #fff;
  border: none;
  padding: 0.6rem 1.5rem;
  border-radius: 4px;
  font-size: 1rem;
  cursor: pointer;
}

form button[type="submit"]:hover {
  background-color: #164869;
}

```

8.2 Jarak Antar Field

```

form p {
  margin-bottom: 1rem;
}

```

Ingat dari [dokumentasi HTML](#), setiap field form dibungkus tag `<p>`. Aturan ini memberi jarak `1rem` di bawah **setiap** `<p>` di dalam form, sehingga tiap field (Judul, Pengarang, dst.) punya jarak yang jelas dari field berikutnya, tidak menempel rapat.

8.3 Label sebagai Blok Tersendiri

```

form label {
  display: block;
  margin-bottom: 0.35rem;
  font-weight: 600;
  color: #444;
}

```

- `display: block;` — secara default, `<label>` adalah elemen *inline* (sejajar dengan teks di sekitarnya, tidak otomatis pindah baris). Mengubahnya jadi `block` memaksa label **selalu memenuhi satu baris penuh**, memastikan input di bawahnya (ingat HTML memakai `<br>` ** setelah label — lihat [dokumentasi HTML §4.3](#)) selalu berada tepat di baris baru.
- `margin-bottom: 0.35rem;` — jarak kecil antara teks label dan kotak input di bawahnya.

- `font-weight: 600;` — label dibuat agak tebal, supaya nama field (misalnya "Judul") lebih menonjol dibanding teks isian biasa, membantu pengguna memindai form dengan cepat.
- `color: #444;` — abu-abu gelap, sedikit lebih terang dari warna teks body (`#2b2b2b` dari [bab 3](#)), memberi variasi warna tanpa kontras berlebihan.

8.4 Kotak Input & Dropdown

```
form input,
form select {
  width: 100%;
  max-width: 400px;
  padding: 0.55rem 0.7rem;
  border: 1px solid #cdd4da;
  border-radius: 4px;
  font-size: 1rem;
}
```

- Selector `form input, form select` (dipisah koma, sama seperti `th, td` di [bab 7](#)) menerapkan gaya yang sama ke **semua** elemen `<input>` (termasuk `type="text"` dan `type="number"`) **dan** elemen `<select>` di dalam form — mencakup semua field yang dibahas di [dokumentasi HTML §4.4](#).
- `width: 100%; max-width: 400px;` — kombinasi ini artinya: lebar input mengikuti lebar kotak pembungkusnya (`100%`), **tapi** tidak akan pernah melebihi `400px`. Di layar sempit (HP), input akan menyempit penuh mengikuti layar; di layar lebar, input tidak akan melebar tak wajar sampai selebar section (`1000px` dari [bab 5](#)).
- `border: 1px solid #cdd4da;` — garis tepi tipis abu-abu muda, menggantikan border bawaan browser yang biasanya lebih tebal/gelap.
- `border-radius: 4px;` dan `padding / font-size` — sudut sedikit membulat dan ruang yang nyaman di dalam kotak, konsisten dengan gaya komponen lain (tombol, kartu) di halaman ini.

Catatan: aturan ini **tidak** menyertakan `box-sizing`, tapi karena di [bab 3](#) sudah ada `* { box-sizing: border-box; }` yang berlaku untuk semua elemen, `padding` dan `border` pada input ini otomatis "dimakan" dari dalam `width: 100% / max-width: 400px`, bukan menambah lebar total — inilah manfaat konkret dari reset di awal file yang sudah dijelaskan di [bab 3](#).

8.5 Tombol Submit

```
form button[type="submit"] {
  background-color: #1d5b8a;
  color: #fff;
  border: none;
```

```
padding: 0.6rem 1.5rem;
border-radius: 4px;
font-size: 1rem;
cursor: pointer;
}

form button[type="submit"]:hover {
  background-color: #164869;
}
```

- `button[type="submit"]` adalah **attribute selector** (lihat [konsep dasar §1.4](#)): memilih elemen `<button>` yang punya atribut `type` bernilai persis `"submit"`. Ini secara khusus menyorot tombol "Simpan" (lihat [dokumentasi HTML §4.5](#)), **tanpa** menyentuh tombol Edit/Hapus di tabel yang bertipe `type="button"` ([dokumentasi HTML §3.2](#)) — contoh nyata kenapa penulisan `type="button"` secara eksplisit di HTML (dibanding membiarkan default) penting: ia menjadi "penanda" yang bisa dipakai CSS untuk membedakan jenis tombol.
- Tombol diberi warna biru tema solid, teks putih, tanpa border, dan kursor pointer — tampil sebagai tombol aksi utama yang jelas.
- `:hover` mengubah warna jadi biru **lebih gelap** (`#164869` dibanding `#1d5b8a`) saat disorot kursor — memberi umpan balik visual yang sama konsepnya dengan `a:hover` ([bab 3](#)) dan `tbody tr:hover` ([bab 7](#)).

Lanjut ke: [CSS: Footer](#)

9. CSS: Footer

Bagian terakhir dan paling sederhana di `style.css`.

9.1 Kode CSS

```
/* ===== Footer ===== */
footer {
  text-align: center;
  padding: 1.25rem;
  color: #7a8794;
  font-size: 0.9rem;
}
```

9.2 Penjelasan

Menata teks copyright di `<footer>` (© 2026 SIMPUS-Mini – Jobsheet 1) yang muncul di semua halaman:

Properti	Efek
<code>text-align: center;</code>	Teks diratakan ke tengah halaman, bukan rata kiri seperti teks pada umumnya — memberi kesan footer sebagai penutup yang "netral", tidak berat sebelah.
<code>padding: 1.25rem;</code>	Jarak seragam di keempat sisi, memberi ruang napas di sekitar teks copyright.
<code>color: #7a8794;</code>	Abu-abu medium — sengaja dibuat lebih pudar/kurang menonjol dibanding warna teks utama (<code>#2b2b2b</code> dari bab 3), karena info copyright memang bukan informasi utama yang perlu menarik perhatian pengguna.
<code>font-size: 0.9rem;</code>	Ukuran teks sedikit lebih kecil dari ukuran normal (<code>1rem</code>), memperkuat kesan bahwa ini teks sekunder/tambahan.

Pola ini — memperkecil ukuran dan memudahkan warna untuk informasi yang kurang penting — adalah teknik hierarki visual yang sama dengan yang dibahas di [bab 6 \(label kartu statistik\)](#): elemen yang secara fungsi kurang penting diberi bobot visual yang lebih ringan, supaya perhatian pengguna terfokus ke konten utama halaman.

Lanjut ke: [Rangkuman & Latihan Lanjutan](#)

10. Rangkuman & Latihan Lanjutan

10.1 Rangkuman Keseluruhan Jobsheet 2

Bagian style.css	Selector Utama	Konsep CSS yang Dipelajari
Reset & Body	<code>*, body, a</code>	CSS reset, <code>box-sizing: border-box</code> , pewarisan gaya (<i>inheritance</i>), pseudo-class <code>:hover</code>
Header & Navbar	<code>header, header nav ul</code>	Flexbox (<code>display: flex</code> , <code>align-items</code> , <code>justify-content</code> , <code>flex-wrap</code> , <code>gap</code>)

Bagian style.css	Selector Utama	Konsep CSS yang Dipelajari
Main & Section	main, section	max-width + margin: auto untuk menengahkan konten, box-shadow, border-radius
Grid Kartu Statistik	main section:nth-of-type(2)	CSS Grid (display: grid, grid-template-columns, fr unit), pseudo-class posisi :nth-of-type
Tabel	table, th, td, tbody tr:nth-child(even)	border-collapse, <i>zebra stripes</i> dengan :nth-child(even), styling tombol dengan :first-of-type / :last-of-type
Form	form label, form input, form select	Attribute selector button[type="submit"], display: block, batas lebar responsif
Footer	footer	Hierarki visual lewat ukuran & warna teks

10.2 Konsep Inti yang Perlu Diingat

1. **CSS terpisah dari HTML** dan dihubungkan lewat `<link rel="stylesheet">` — satu file CSS bisa dipakai ulang di banyak halaman HTML sekaligus ([bab 1](#), [bab 2](#)).
2. `box-sizing: border-box` membuat perhitungan lebar/tinggi elemen jauh lebih mudah diprediksi ketika ada padding / border ([bab 3](#)).
3. **Flexbox** untuk tata letak 1 dimensi (navbar), **CSS Grid** untuk tata letak 2 dimensi berbasis kolom/baris (kartu statistik) — dua alat berbeda untuk kebutuhan berbeda ([bab 4](#), [bab 6](#)).
4. **Pseudo-class** (:hover, :nth-child, :nth-of-type, :first-of-type, :last-of-type) memungkinkan styling berdasarkan **state** (kondisi kursor) atau **posisi** elemen, tanpa perlu menambah atribut apa pun di HTML.
5. **Spesifisitas** menentukan aturan mana yang menang kalau ada dua aturan CSS yang menyasar elemen yang sama — selector lebih spesifik (lebih banyak "syarat") umumnya menang, kecuali kalau spesifisitasnya sama, di mana **urutan penulisan di file** yang menentukan ([bab 4 §4.7](#), [bab 7 §7.5](#)).

10.3 Cara Mencoba Sendiri

1. Buka `index.html` di browser dan bandingkan dengan tampilan `index.html` jobsheet-01 (tanpa CSS) — perhatikan navbar sekarang sejajar horizontal, dan kartu statistik tersusun 3 kolom.

2. Buka *DevTools* browser (klik kanan → *Inspect/Inspeksi*, atau tombol `F12`), klik tab **Elements/Inspector**, lalu klik salah satu elemen (misalnya kartu "Total Buku"). Panel di sebelahnya akan menampilkan aturan CSS mana saja yang berlaku pada elemen itu — cara terbaik untuk **melihat langsung** hubungan antara selector CSS dan tampilannya.
3. Di *DevTools*, coba ubah salah satu nilai CSS secara langsung (misalnya ubah `grid-template-columns: repeat(3, 1fr)` menjadi `repeat(2, 1fr)`) dan lihat efeknya seketika — perubahan ini **sementara**, hanya di browser, tidak mengubah file aslinya, jadi aman untuk bereksperimen.

10.4 Ide Latihan Tambahan (Opsional)

1. **Ubah skema warna** — ganti nilai `#1d5b8a` (warna biru tema) di seluruh file `style.css` dengan warna lain, misalnya hijau tua, lalu amati bagaimana warna itu konsisten muncul di header, judul section, tombol submit, dan header tabel — karena semuanya memakai nilai hex yang sama.
2. **Tambah kolom keempat** di grid kartu statistik — tambahkan satu `<article>` baru di HTML (misalnya "Buku Terlambat"), lalu ubah `repeat(3, 1fr)` menjadi `repeat(4, 1fr)` di CSS.
3. **Buat tombol ketiga di tabel** — tambahkan tombol "Detail" di antara Edit dan Hapus pada `buku/list.html`, lalu amati apakah warnanya sesuai harapan (ingat catatan di [bab 7 §7.6](#) tentang `:first-of-type / :last-of-type` yang berbasis posisi, bukan makna). Coba perbaiki dengan memberi `class` khusus jika warnanya tidak sesuai.
4. **Uji responsivitas sederhana** — perkecil lebar jendela browser secara bertahap sampai sangat sempit (seperti lebar HP), amati kapan `flex-wrap: wrap` pada navbar mulai memindahkan menu ke baris baru.

Kalau ada bagian yang masih membingungkan, coba baca ulang [konsep dasar CSS di bab 1](#) — sebagian besar istilah teknis di bab-bab lain dijelaskan di sana.